Lumen

Local lighting and sensing platform on ESP32

- local first
- MQTT today
- no cloud required
- built to grow without turning into a mess

What this is

Lumen is not a cloud toy and it is not a one-off classroom demo.

It is a local node that:

- reads the environment
- controls LED output
- accepts commands
- reports what it is doing
- keeps enough structure that the firmware can grow without collapsing into one giant loop

That is the whole point.

Code: `src/lumen_main.cpp`, `src/lumen_task_manager.cpp`, `src/lumen_mqtt_client.cpp`

Why this exists

A lot of systems like this get ruined by two bad choices:

1. add cloud dependence for no good reason
2. keep piling logic into one shared loop until nobody wants to touch it

Lumen avoids both.

Local control is simpler, faster, easier to debug, and more useful in places where internet is weak, expensive, or irrelevant. A farm does not need a SaaS dependency just to read a sensor or switch a light.

Food and lighting are physical problems. They should stay local unless there is a real reason not to.

What users can do today

Monitor

- ambient light
- temperature
- humidity
- node state
- online / offline

Control

- LED brightness
- LED channel mix
- operating mode

Receive

- telemetry
- status updates
- acknowledgements
- energy data

This is not hypothetical. The current firmware already does it.

Code: `src/lumen_sensor_manager.cpp`, `src/lumen_led_control.cpp`, `src/lumen_energy_tracker.cpp`, `src/lumen_mqtt_client.cpp`

Where it fits

Home growing

Shelves, tents, and small enclosures.

Farms and greenhouses

Local control where internet is weak or unnecessary.

Hobby and platform work

ESP32, sensors, LEDs, and a base you can keep extending.

One future direction is spectrum-aware plant lighting. That overlaps with photomorphogenesis, where light intensity, timing, and composition all matter.

Current implementation

No hand-waving. This is the current state.

Done

- BH1750 ambient light sensing
- DHT11 temperature and humidity sensing
- LED control
- autonomous brightness behavior
- manual control
- MQTT telemetry and commands
- energy tracking
- config persistence

Not done yet

- TCS34725 support
- RGB/Clear runtime data
- spectrum-aware control logic
- photoperiod scheduling
- direct transport mode
- multi-node coordination

Code: `src/lumen_sensor_manager.cpp`, `src/lumen_led_control.cpp`, `src/lumen_config_manager.cpp`

Hardware profile

Current hardware is simple on purpose.

- controller:
  - ESP32 / ESP32-WROOM family
- sensors:
  - BH1750
  - DHT11
- actuation:
  - WS2812B / NeoPixel-compatible LEDs
- next addition:
  - TCS34725

center

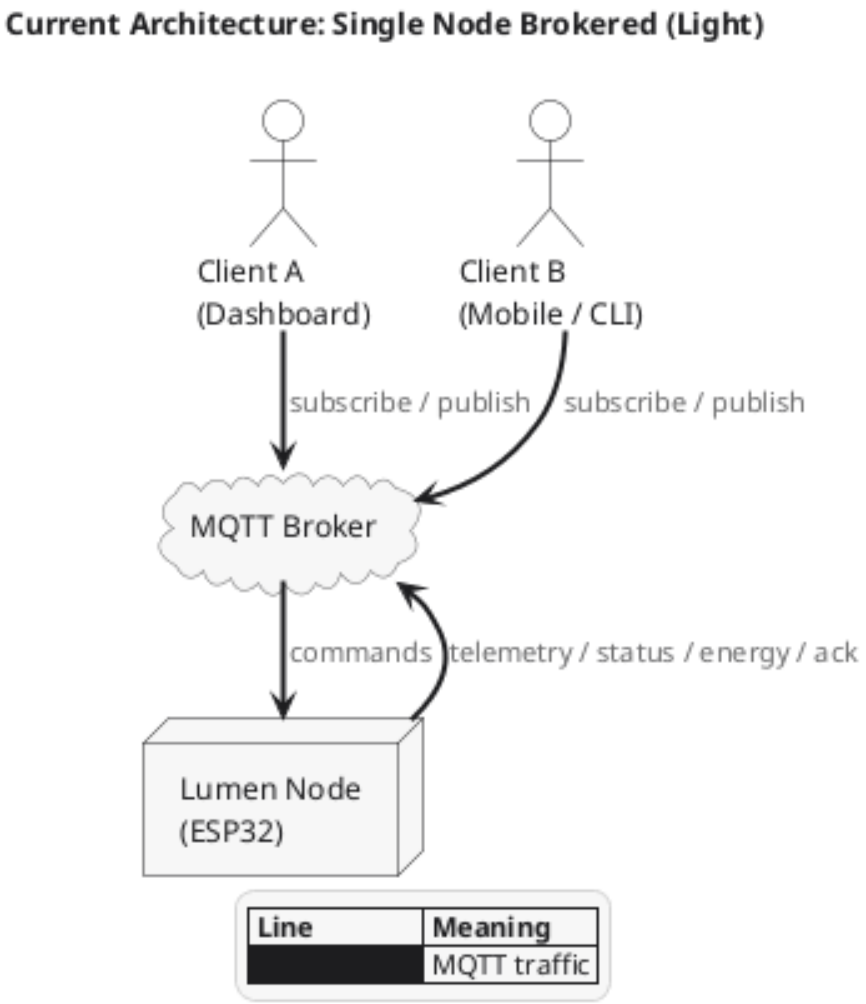
Config: `include/lumen_board_config.h` Wiring: `WIRING.md`

Current architecture

This is the current production shape.

- one node
- brokered transport
- one or many clients through the broker
- clean topic namespace
- no cloud requirement

Code: `src/lumen_mqtt_client.cpp`, `src/lumen_wifi_manager.cpp`



Firmware design

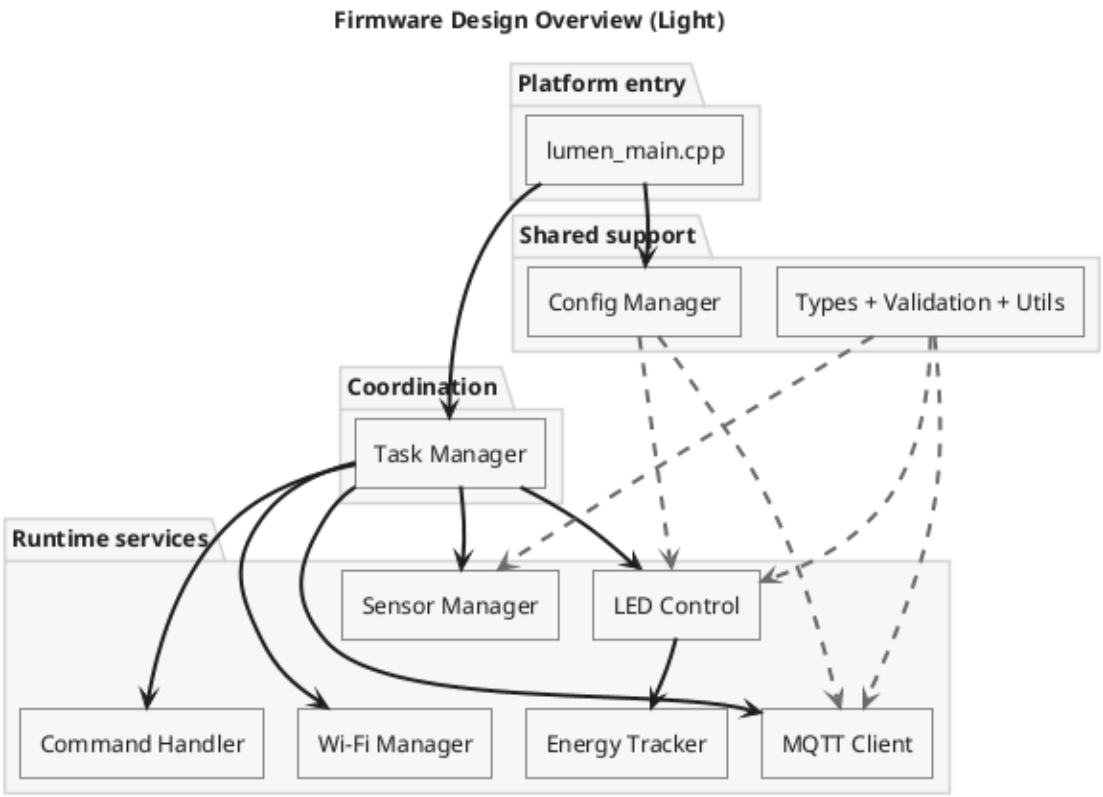
The firmware is split by responsibility because stuffing everything into one loop is how projects become unmaintainable.

Core pieces:

- platform entry
- shared support
- connectivity
- sensing
- control
- coordination
- persistence

The split is deliberate. It keeps sensing, control, transport, and storage from tripping over each other.

Code map: `src/lumen_main.cpp`, `src/lumen_task_manager.cpp`, `src/lumen_config_manager.cpp`, `src/lumen_system_utils.cpp`, `src/lumen_type_validation.cpp`



Runtime flow

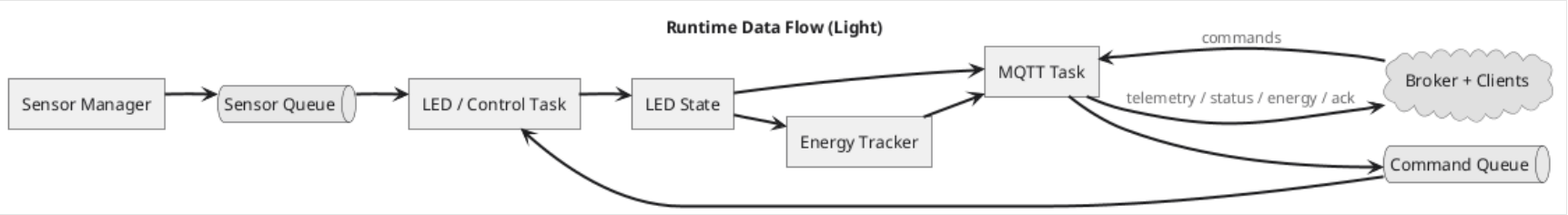
Sensor path

- sensing
- control
- state update
- telemetry

Command path

- MQTT inbound
- command queue
- control task
- acknowledgement

These parts have different timing and different failure modes. That separation matters.



Code: `void taskSensors( ... )`, `void taskLed( ... )`, `void taskMqtt( ... )` in `src/lumen_task_manager.cpp`

Code, not hand-waving

This is the actual shape of the runtime. The tasks are real.

```
void taskSensors(void* parameter);
void taskLed(void* parameter);
void taskMqtt(void* parameter);
```

And the MQTT task does exactly what you would expect:

```
const bool wifi_ready = WifiManager::connectOrPoll();
const bool mqtt_ready = MqttClient::connectOrPoll();
```

If both are up, it publishes telemetry and drains the outbound queues.

Source: `src/lumen_task_manager.cpp`

Control path in code

Sensor reads and lighting control are also straightforward.

```
bool readCurrent(SensorReading& out);
void controlTick(const SensorReading& latest_reading);
AckMessage applyCommand(const CommandEnvelope& command);
```

That gives you three clear control points:

- read the world
- update the light
- apply direct commands

Nothing mystical here.

Source: `src/lumen_sensor_manager.cpp` and `src/lumen_led_control.cpp`

Transport path in code

The MQTT side is not magic either.

```
bool connectOrPoll();
bool publishTelemetry(const SensorReading& reading);
bool publishStatus(const StatusMessage& status);
bool publishEnergy(const EnergyMessage& energy);
bool publishAck(const AckMessage& ack);
```

So when the deck says telemetry, status, energy, and acknowledgements are separate paths, that is not a marketing phrase. Those functions exist in the code.

Source: `src/lumen_mqtt_client.cpp`

Technical decisions

Why local first

- no cloud dependency
- lower operational friction
- works in remote setups
- easier to debug

Why MQTT

- natural fit for telemetry plus commands
- multiple clients are easy
- keeps transport clean

Code: `src/lumen_mqtt_client.cpp`, `src/lumen_task_manager.cpp`

Why MsgPack

- smaller payloads
- less wire overhead
- better fit for constrained firmware

Why tasks and queues

- cleaner timing boundaries
- less coupling
- easier to reason about than one big shared-state loop

Sensor choices

The current stack is not random.

- **BH1750 now**
  - enough for brightness-based control
  - simple and stable
- **DHT11 now**
  - cheap
  - common
  - enough for baseline environmental context
- **TCS34725 next**
  - adds RGB and clear data
  - moves the system toward spectrum awareness

Brightness is not the same thing as composition. That is why BH1750 and TCS34725 are different jobs.

Wiring and deployment

The physical setup is straightforward.

- SDA: GPIO21
- SCL: GPIO22
- DHT11 DATA: GPIO4
- LED DATA: GPIO18
- sensors on 3.3 V
- LEDs on regulated 5 V
- common ground everywhere

And yes, it is local-only in the cloud sense.

```
Lumen node ⟷ local Wi-Fi ⟷ local MQTT broker ⟷ local client/frontend
```

Config: `include/lumen_board_config.h` Wiring: `WIRING.md`

What is missing

The current firmware is useful, but it is still narrow.

Current limits:

- single-node implementation
- MQTT-only transport
- no spectral sensing yet
- no direct mode yet
- RGB hardware only approximates far-red

That is fine. Better to be honest than pretend a prototype is a finished system.

Roadmap

The platform is going in two directions.

Deeper on one node

- TCS34725 integration
- richer sensor model
- spectrum-aware control
- photoperiod support
- better horticulture-focused behavior

Broader across deployments

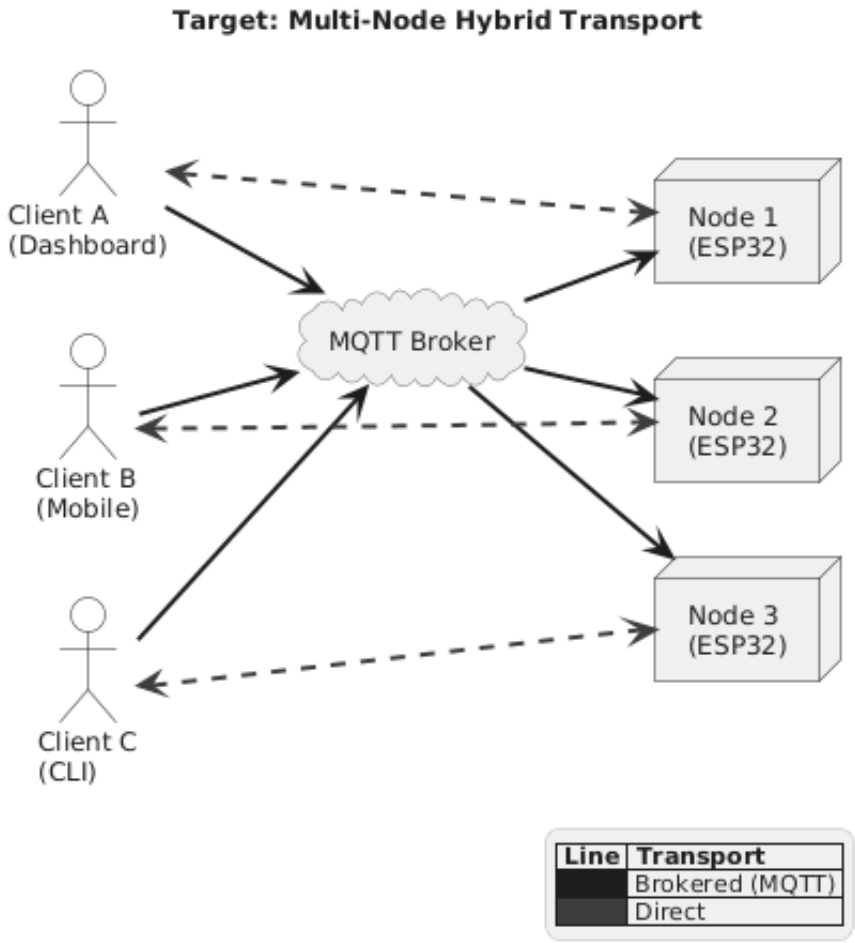
- more than one node
- direct mode
- hybrid mode
- cleaner client integration

That is the roadmap. Not magic. Just the next sensible steps.

Target architecture

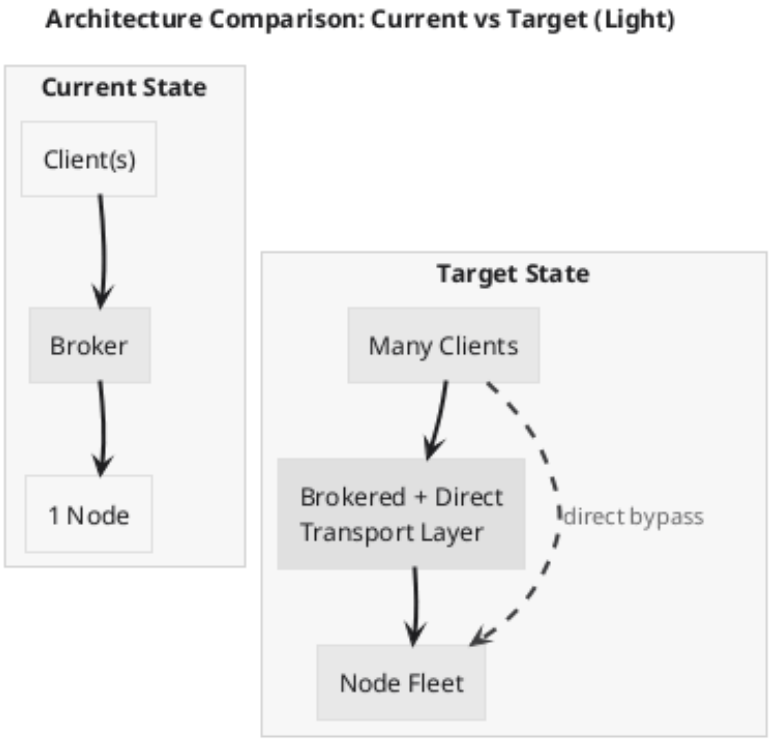
Current architecture is fine for now. This is where it goes next.

- many clients
- brokered and direct paths
- many nodes
- cleaner scaling model



Current vs target

This is the whole direction in one slide.



Dimension	Current	Target
Clients	One or many via broker	Many
Nodes	Single	Fleet
Transport	MQTT only	MQTT + Direct + Hybrid
Discovery	Manual	Automated
Coordination	None	Group / Fleet level

MQTT model

Lumen currently uses MQTT as the transport layer.

Published topics:

```
lumen/<device_id>/telemetry
lumen/<device_id>/status
lumen/<device_id>/energy
lumen/<device_id>/availability
lumen/<device_id>/ack/<command_id>
```

Subscribed topics:

```
lumen/<device_id>/command/led
lumen/<device_id>/command/config
lumen/<device_id>/command/mode
```

Payloads use MsgPack. Availability uses plain string values.

Source: `include/lumen_board_config.h`, `src/lumen_mqtt_client.cpp`

Closing

Lumen already works as a local lighting and sensing node.

What matters now is not pretending it is finished. What matters is that the base is solid:

- local first
- structured firmware
- real sensing
- real control
- room to grow

That is a good place to be.